

useEffect Hook - Complete Notes

Table of Contents

1. What is useEffect?
 2. Why Do We Need useEffect?
 3. Basic Syntax
 4. How useEffect Works
 5. Dependency Array Explained
 6. Common Use Cases
 7. Cleanup Function
 8. Important Rules
 9. Common Mistakes to Avoid
 10. Interview Questions & Answers
-

What is useEffect?

useEffect is a React Hook that lets you perform **side effects** in functional components.

What are Side Effects?

Side effects are operations that affect things outside the component, such as:

- Fetching data from an API
- Updating the DOM directly
- Setting up subscriptions or timers
- Reading from localStorage
- Logging to console

Simple Analogy: Think of useEffect as a way to tell React: "Hey, after you finish rendering this component, I need you to do this extra task."

Why Do We Need useEffect?

In class components, we had lifecycle methods like:

- `componentDidMount()` - runs after component appears
- `componentDidUpdate()` - runs after component updates
- `componentWillUnmount()` - runs before component disappears

useEffect combines all three lifecycle methods into one Hook!

Basic Syntax

```
import { useEffect } from 'react';

useEffect(() => {
  // Your side effect code here

  return () => {
    // Cleanup code (optional)
  };
}, [dependencies]); // Dependency array (optional)
```

Three Parts:

1. **Effect Function** - The code that runs
 2. **Cleanup Function** (optional) - Runs before the next effect or when component unmounts
 3. **Dependency Array** (optional) - Controls when the effect runs
-

How useEffect Works

The Flow:

1. React renders your component
2. Browser paints the screen (user sees the UI)
3. useEffect runs **after** the paint
4. If dependencies change, useEffect runs again

Important: useEffect doesn't block the browser from updating the screen, so your app feels fast!

Dependency Array Explained

The dependency array is **the most important part** of useEffect. It controls when your effect runs.

1. No Dependency Array

```
useEffect(() => {
  console.log('Runs after EVERY render');
});
```

- Runs after every single render
- Usually **not recommended** (can cause performance issues)

2. Empty Dependency Array

```
useEffect(() => {
  console.log('Runs only ONCE after first render');
}, []);
```

- Runs only once when component mounts
- Like `componentDidMount()`
- Perfect for: fetching initial data, setting up subscriptions

3. With Dependencies

```
useEffect(() => {
  console.log('Runs when count or name changes');
}, [count, name]);
```

- Runs after first render AND whenever `count` or `name` changes
- React compares old and new values
- Only re-runs if values actually changed

Common Use Cases

1. Fetching Data from API

```
import { useState, useEffect } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Fetch user data when userId changes
    setLoading(true);

    fetch(`https://api.example.com/users/${userId}`)
      .then(response => response.json())
      .then(data => {
        setUser(data);
        setLoading(false);
      });
  }, [userId]); // Re-fetch when userId changes

  if (loading) return <p>Loading...</p>;
  return <div>{user.name}</div>;
}
```

2. Setting Up a Timer

```
function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);

    // Cleanup: clear interval when component unmounts
    return () => clearInterval(interval);
  }, []); // Empty array = runs once

  return <div>Seconds: {seconds}</div>;
}
```

3. Updating Document Title

```
function PageTitle({ title }) {
  useEffect(() => {
    document.title = title;
  }, [title]); // Update title when it changes

  return <h1>{title}</h1>;
}
```

4. LocalStorage Example

```
function Counter() {
  const [count, setCount] = useState(() => {
    // Initialize from localStorage
    const saved = localStorage.getItem('count');
    return saved ? JSON.parse(saved) : 0;
  });

  useEffect(() => {
    // Save to localStorage whenever count changes
    localStorage.setItem('count', JSON.stringify(count));
  }, [count]);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

5. Event Listeners

```
function WindowSize() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => {
      setWidth(window.innerWidth);
    };

    window.addEventListener('resize', handleResize);

    // Cleanup: remove listener when component unmounts
    return () => {
      window.removeEventListener('resize', handleResize);
    };
  }, []); // Empty array = set up once

  return <p>Window width: {width}px</p>;
}
```

Cleanup Function

The cleanup function is returned from `useEffect` and runs:

1. Before the effect runs again (if dependencies changed)
2. When the component unmounts (disappears from screen)

Why Do We Need Cleanup?

Without cleanup, you can have:

- **Memory leaks** (timers keep running)
- **Multiple event listeners** (slowing down app)
- **Stale data** (old subscriptions still active)

Cleanup Example:

```
useEffect(() => {
  // Setup
  const timer = setTimeout(() => {
    console.log('Hello!');
  }, 1000);

  // Cleanup
  return () => {
    clearTimeout(timer);
  };
}, []);
```

When Cleanup Runs:

```
useEffect(() => {
  console.log('Effect runs');

  return () => {
    console.log('Cleanup runs BEFORE next effect');
  };
}, [count]);

// Flow when count changes:
// 1. Cleanup runs (from previous effect)
// 2. Effect runs (with new count value)
```

Important Rules

1. Only Call Hooks at the Top Level

❌ Don't do this:

```
if (condition) {
  useEffect(() => { }); // Wrong!
}
```

✅ Do this:

```
useEffect(() => {
  if (condition) {
    // Your code here
  }
}, [condition]);
```

2. Only Call Hooks in React Functions

- Call in functional components
- Call in custom hooks
- Don't call in regular JavaScript functions

3. Include All Dependencies

If your effect uses a variable or prop, include it in the dependency array.

❌ Don't do this:

```
function SearchResults({ query }) {
  useEffect(() => {
    fetchResults(query); // Uses query but not in dependencies
  }, []); // Missing query!
}
```

✅ **Do this:**

```
function SearchResults({ query }) {
  useEffect(() => {
    fetchResults(query);
  }, [query]); // Include query
}
```

Common Mistakes to Avoid

Mistake 1: Infinite Loop

```
// ❌ BAD - Creates infinite loop!
const [count, setCount] = useState(0);

useEffect(() => {
  setCount(count + 1); // Changes count
}); // No dependency array, runs after every render!

// ✅ GOOD - Controlled update
useEffect(() => {
  // Only run once or based on specific condition
  setCount(10);
}, []); // Empty array = runs once
```

Mistake 2: Missing Dependencies

```
// ❌ BAD
function SearchBox() {
  const [query, setQuery] = useState('');

  useEffect(() => {
    fetchData(query); // Uses query
  }, []); // Missing query in dependencies!
}

// ✅ GOOD
useEffect(() => {
  fetchData(query);
}, [query]); // Include query
```

Mistake 3: Forgetting Cleanup

```
// ❌ BAD - Memory leak!
useEffect(() => {
  const timer = setInterval(() => {
    console.log('Running...');
  }, 1000);
  // No cleanup! Timer keeps running even after component unmounts
}, []);

// ✅ GOOD
useEffect(() => {
  const timer = setInterval(() => {
    console.log('Running...');
  }, 1000);

  return () => clearInterval(timer); // Cleanup
}, []);
```

Mistake 4: Using Async Directly in useEffect

```
// ❌ BAD
useEffect(async () => {
  const data = await fetchData(); // Can't use async directly
}, []);

// ✅ GOOD - Option 1: Create async function inside
useEffect(() => {
  const loadData = async () => {
    const data = await fetchData();
    setData(data);
  };

  loadData();
}, []);

// ✅ GOOD - Option 2: Use .then()
useEffect(() => {
  fetchData()
    .then(data => setData(data));
}, []);
```

Interview Questions & Answers

Q1: What is useEffect and why do we use it?

Answer: useEffect is a React Hook that allows us to perform side effects in functional components. Side effects are operations like fetching data, updating the DOM, setting up subscriptions, or timers. We use it because functional components don't have lifecycle methods like class components, and useEffect combines componentDidMount, componentDidUpdate, and componentWillUnmount into one API.

Q2: Explain the dependency array in useEffect.

Answer: The dependency array is the second argument to `useEffect` and controls when the effect runs:

- **No array:** Effect runs after every render
- **Empty array** `[]`: Effect runs only once after initial render
- **With values** `[a, b]`: Effect runs after initial render and whenever `a` or `b` changes

React compares the dependencies using shallow comparison to decide if the effect should re-run.

Q3: What is the cleanup function and when is it used?

Answer: The cleanup function is a function returned from `useEffect`. It runs before the component unmounts or before the effect runs again. We use it to clean up side effects like:

- Clearing timers or intervals
- Removing event listeners
- Cancelling API requests
- Unsubscribing from subscriptions

This prevents memory leaks and ensures our app doesn't have unwanted behavior.

Q4: Can you use multiple useEffect hooks in one component?

Answer: Yes! You can use multiple `useEffect` hooks to separate different concerns. This makes code more organized and easier to maintain.

```
function UserProfile({ userId }) {  
  // Effect 1: Fetch user data  
  useEffect(() => {  
    fetchUser(userId);  
  }, [userId]);  
  
  // Effect 2: Update page title  
  useEffect(() => {  
    document.title = `User ${userId}`;  
  }, [userId]);  
  
  // Effect 3: Log analytics  
  useEffect(() => {  
    logPageView();  
  }, []);  
}
```

Q5: How do you handle async operations in useEffect?

Answer: You cannot make `useEffect` itself async, but you can create an async function inside it:

```
useEffect(() => {
  const fetchData = async () => {
    try {
      const response = await fetch(url);
      const data = await response.json();
      setData(data);
    } catch (error) {
      console.error(error);
    }
  };

  fetchData();
}, [url]);
```

Q6: What happens if you don't provide a dependency array?

Answer: If you don't provide a dependency array, the effect runs after every render of the component. This can cause performance issues and infinite loops if the effect triggers a state update, which causes a re-render, which runs the effect again, and so on.

Q7: Difference between `useEffect` and `useLayoutEffect`?

Answer:

- **useEffect:** Runs **after** the browser paints the screen (asynchronous). Used for most side effects.
- **useLayoutEffect:** Runs **before** the browser paints (synchronous). Used when you need to measure or mutate the DOM and want to avoid visual flickering.

In 99% of cases, use `useEffect`. Only use `useLayoutEffect` if you see visual problems.

Q8: How do you prevent infinite loops in `useEffect`?

Answer: Infinite loops happen when:

1. The effect updates state that triggers a re-render
2. The effect has no dependency array or wrong dependencies

Prevention:

```
// ✅ Use dependency array
useEffect(() => {
  setCount(c => c + 1);
}, []); // Runs only once

// ✅ Use functional updates if you need current state
useEffect(() => {
  setCount(c => c + 1); // Doesn't depend on count
}, [somethingElse]);
```

Q9: Can you give an example of fetching data with useEffect?

Answer:

```
function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchUsers = async () => {
      try {
        setLoading(true);
        const response = await fetch('https://api.example.com/users');

        if (!response.ok) throw new Error('Failed to fetch');

        const data = await response.json();
        setUsers(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchUsers();
  }, []); // Runs once on mount

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <ul>{users.map(user => <li key={user.id}>{user.name}</li>)}</ul>;
}
```

Q10: What's the execution order in useEffect?

Answer:

1. Component renders (returns JSX)
2. React updates the DOM
3. Browser paints the screen (user sees UI)
4. useEffect runs
5. If dependencies change → cleanup runs → effect runs again

```
function Example() {
  console.log('1. Component renders');

  useEffect(() => {
    console.log('3. Effect runs');

    return () => {
      console.log('2. Cleanup runs (on next render or unmount)');
    };
  });

  return <div>Hello</div>; // This shows first
}
```

Quick Reference Cheat Sheet

```
// Run once on mount
useEffect(() => {
  // Code here
}, []);

// Run on every render
useEffect(() => {
  // Code here
});

// Run when dependencies change
useEffect(() => {
  // Code here
}, [dep1, dep2]);

// With cleanup
useEffect(() => {
  // Setup
  return () => {
    // Cleanup
  };
}, [deps]);

// Async inside useEffect
useEffect(() => {
  const doAsync = async () => {
    const result = await fetchData();
  };
  doAsync();
}, []);
```

Best Practices

1.  Always include all dependencies that are used inside the effect
2.  Use cleanup function for subscriptions, timers, and event listeners
3.  Keep effects focused on one concern (use multiple effects if needed)
4.  Extract complex logic into custom hooks

5. ✅ Handle loading and error states when fetching data
 6. ❌ Don't update state without conditions (can cause infinite loops)
 7. ❌ Don't forget to cleanup to prevent memory leaks
 8. ❌ Don't make useEffect callback async directly
-

Summary

- **useEffect** lets you perform side effects in functional components
 - It runs **after** the component renders and paints to the screen
 - The **dependency array** controls when the effect runs
 - The **cleanup function** prevents memory leaks and removes old effects
 - You can use **multiple useEffect hooks** in one component
 - Always include **all dependencies** to avoid bugs
 - Use **cleanup** for timers, event listeners, and subscriptions
-

Good luck with your interviews! 🚀